

Jet Flavour Tagging con Transformer (GN2)

Implementazione in PyTorch

ATLAS Collaboration — *Nat. Commun.* **17**, 541 (2026)

Luca Nasini (656244)

20 maggio 2026

Obiettivo: addestrare un transformer a classificare i jet di collisioni pp come b , c , τ o *light* a partire dalle tracce di particelle cariche associate al jet

Perché il Jet Flavour Tagging?

Contesto fisico

- I jet adronici sono gli oggetti più abbondanti nelle collisioni pp all'LHC
- Identificare il sapore del quark d'origine è fondamentale per studiare il Modello Standard

Quattro classi

Classe	Descrizione
b -jet	contiene un adrone- b
c -jet	contiene un adrone- c
τ -jet	decadimento adronico di τ
light-jet	quark leggero o gluone

Approcci al flavour tagging

Pipeline tradizionale (a due stadi):

- 1 Algoritmi di basso livello $\rightarrow$ ricostruiscono i vertici spostati
- 2 Classificatore multivariato ad alto livello (e.g. DL1d) $\rightarrow$ combina gli output

GN2: un unico transformer *end-to-end*

Confronto: DL1d vs. GN2

Miglioramento di un fattore 1.5 – 4

Dataset e Preprocessing (preprocess.py)

Dati di input (HDF5, simulazione MC $t\bar{t}$)

- **5.6 milioni** di jet totali, 4 categorie di sapore
- **Livello jet:** p_T, η
- **Livello traccia:** d_0, z_0 , significanze, variabili angolari; flag valid per le tracce utilizzabili

Selezione cinematica

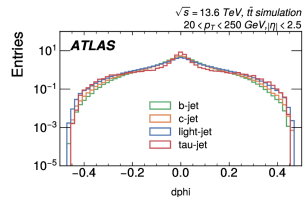
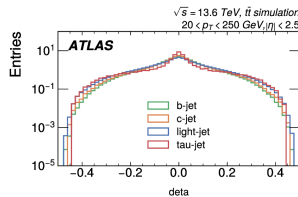
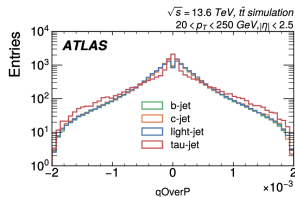
$$20 < p_T < 250 \text{ GeV} \quad \& \quad |\eta| < 2.5$$

Normalizzazione senza data leakage

Media e deviazione standard calcolate
esclusivamente sul training set
Statistiche salvate in norm_stats.json

Splitting (per indice casuale)

Train 70% / Validation 20% / Test 10%
Indici salvati in XXX_indices.npy



Caricamento Efficiente dei Dati (dataset.py)

Tre componenti principali

- GN2Dataset: padding/crop a 40 tracce, normalizzazione, lock su HDF5 per evitare race condition
- _IndexDataset: restituisce solo gli indici dei campioni → garantendo accessi ordinati e contigui su disco
- _BatchCollator: ordina i jet in un batch per letture HDF5 contigue → riducendo l'overhead di I/O

Gestione del padding

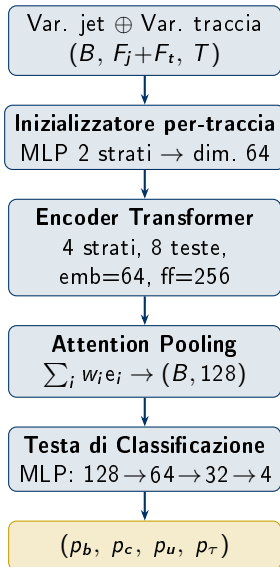
Ogni jet ha un numero variabile di tracce. Le posizioni non valide vengono:

- azzerate con maschera booleana durante la lettura
- escluse dall'attenzione del transformer tramite la stessa maschera

Risultato

gn2_data_loader costruisce un DataLoader PyTorch con batch ordinati e ottimizzati per il training

Architettura del Modello (model.py)



1 - Inizializzatore per-traccia

Concatenazione di tutte le variabili (jet + track). Un MLP a due strati mappa il vettore combinato in uno spazio di dimensione 64.

2 - Encoder Transformer (Pre-LayerNorm)

Per ogni layer ℓ : $x' = x + \text{MHA}(\text{LN}(x))$, $x'' = x' + \text{FFN}(\text{LN}(x'))$

3 - Attention Pooling

$$w_i = \text{softmax}(q^\top e_i) \rightarrow z = \sum_i w_i e_i$$

Jet completamente mascherati ricevono pesi uniformi.

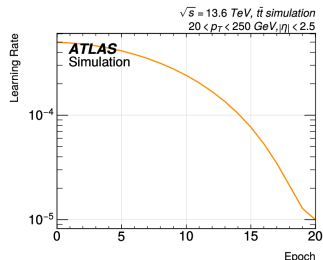
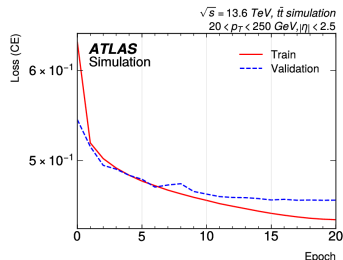
$\Rightarrow \sim 230\,000$ parametri addestrabili

Loss, ottimizzatore e learning rate schedule

Loss: cross-entropy a 4 classi (`ignore_index=-1`)

Ottimizzatore: AdamW, weight decay $\lambda = 10^{-5}$

- 1 Warmup lineare: $10^{-7} \rightarrow 5 \times 10^{-4}$ (primo 1% degli step)
- 2 Cosine annealing: $5 \times 10^{-4} \rightarrow 10^{-5}$



Iperparametri principali

Iperparametro	Valore
Epoche	21
Batch size	2048
Max tracce	40
Attivazione	ReLU
Dropout	No
Grad. clipping	$\ \nabla\ \leq 1.0$
Dispositivo	GPU Tesla T4
Tempo training	~ 17 ore

Discriminanti di Tagging e Curve ROC (evaluate.py)

Discriminante b -tagging (D_b)

$$D_b = \log \left(\frac{p_b}{f_c p_c + f_\tau p_\tau + (1 - f_c - f_\tau) p_u} \right)$$

Parametri ATLAS: $f_c = 0.20$, $f_\tau = 0.05$

Discriminante c -tagging (D_c)

$$D_c = \log \left(\frac{p_c}{f_b p_b + f_\tau p_\tau + (1 - f_b - f_\tau) p_u} \right)$$

Parametri ATLAS: $f_b = 0.30$, $f_\tau = 0.01$

Curve ROC: rigetto vs. efficienza

Per ogni classe di fondo k e soglia θ :

$$R_k(\theta) = \frac{|\text{fondo}_k|}{|\{D > \theta\} \cap \text{fondo}_k|}$$

Rigetto maggiore a pari efficienza = tagger migliore

Output prodotti

- metrics.json
- confusion_matrix.pdf
- score_distributions.pdf
- roc_db.pdf, roc_dc.pdf

Distribuzioni degli Score e Matrice di Confusione

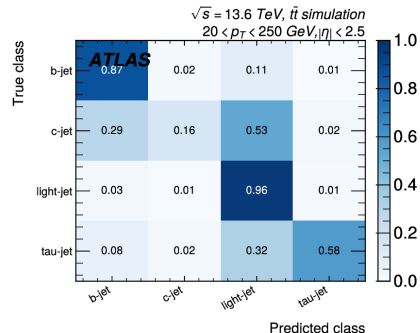
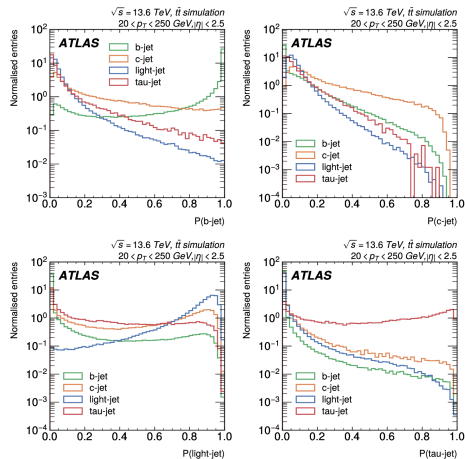


Figura: Matrice di confusione (normalizzata)

Figura: Score softmax $P(\text{classe})$

Metriche sul test set

Classe	P	R	F1	Acc. glob.
<i>b</i> -jet	0.88	0.86	0.88	0.84
<i>c</i> -jet	0.54	0.16	0.24	
light-jet	0.83	0.96	0.89	
τ -jet	0.69	0.58	0.63	

Cause del basso F1 sui *c*-jet

- Vita media intermedia dei mesoni charm
⇒ vertice secondario ambiguo
- Dataset fortemente sbilanciato: *c*-jet e τ -jet sottorappresentati

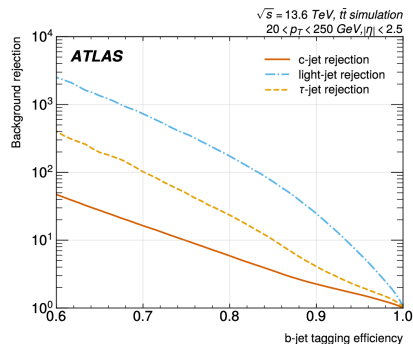
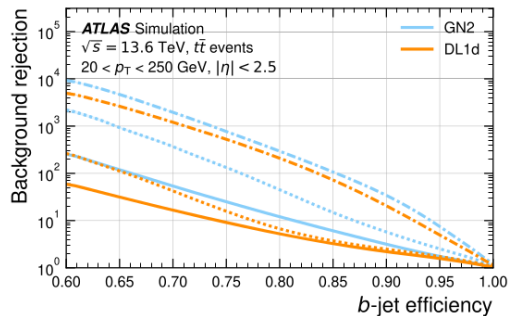
Rigetto del fondo al 70% di eff. *b*-tagging

Fondo	GN2 (ATLAS)	Questa impl.
Jet- <i>c</i>	50	12
Jet leggero	2000	700
Jet- τ	400	100

Rigetto del fondo al 30% di eff. *c*-tagging

Fondo	GN2 (ATLAS)	Questa impl.
Jet- <i>b</i>	20	12
Jet leggero	300	70
Jet- τ	60	4

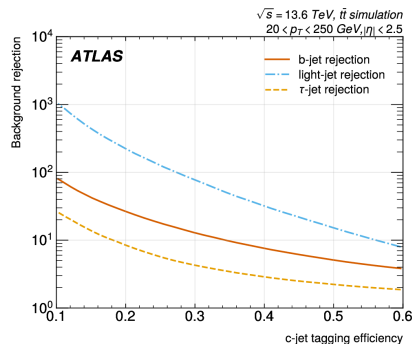
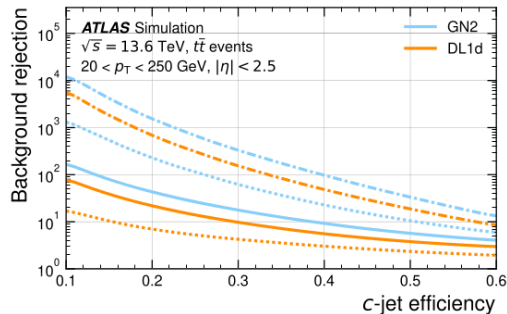
Curve ROC: risultati e confronto (b -tagging)



Interpretazione

Le discrepanze sono ragionevoli con l'architettura ridotta ($\sim 230\,000$ vs. milioni di parametri GN2), il dataset più piccolo e l'assenza di ottimizzazione sistematica degli iperparametri

Curve ROC: risultati e confronto (c-tagging)



Interpretazione

Le discrepanze sono ragionevoli con l'architettura ridotta ($\sim 230\,000$ vs. milioni di parametri GN2), il dataset più piccolo e l'assenza di ottimizzazione sistematica degli iperparametri

Cosa è stato implementato

- ✓ Pipeline completa: preprocess $\rightarrow$ dataset $\rightarrow$ train $\rightarrow$ evaluate
- ✓ Architettura GN2 + ottimizzazione I/O

Sviluppi futuri

- Obiettivi ausiliari (origine tracce + vertici)
- Ottimizzazione degli iperparametri
- Training su dataset completo

Risultati principali

Il modello riproduce **qualitativamente** il comportamento atteso:

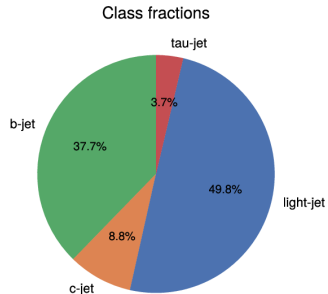
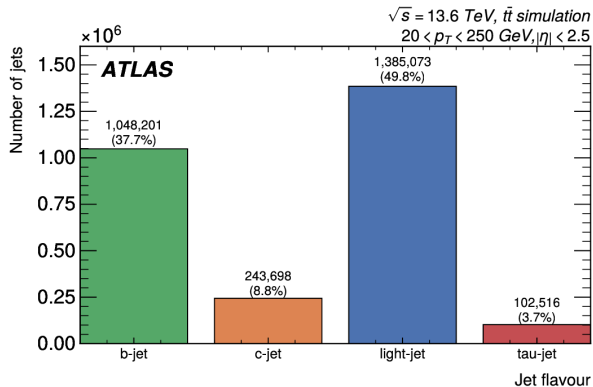
- Buona separazione dei b -jet e light-jet
- Difficoltà strutturale su c -jet e τ -jet

Prestazioni quantitative inferiori a GN2 di un fattore $\sim 3 - 4$ (b -tagging) e $\sim 5 - 10$ (c -tagging)

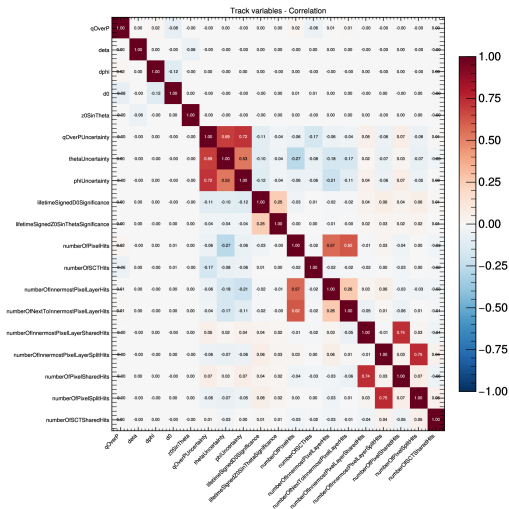
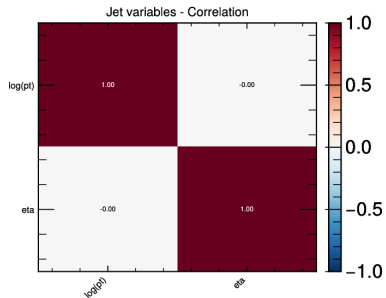
Limitazione principale

I due **obiettivi di addestramento ausiliari** (classificazione dell'origine delle tracce e raggruppamento per vertice comune) potrebbero contribuire fino al 30% di miglioramento nel rigetto del fondo

Appendice: distribuzione jet flavour



Appendice: correlazioni tra feature



Appendice: discriminanti

